

Information Flow Control and Declassification in Programming Languages

Annotated Bibliography

Logan Green
University of Michigan

April 17, 2026

1 Narrative Summary

1.1 Problem Space and Motivation

Information flow control (IFC) studies how information moves through a program and whether that movement is secure. This problem is broader than standard access control. Access control can restrict who is allowed to read or write a file, but it does not by itself prevent information from being leaked indirectly during computation. A program might read a secret value, branch on it, and then reveal the result through a public output, even if no direct read of the secret is allowed. IFC therefore asks not only who may access data, but also where sensitive information is allowed to flow after it has been accessed. Another useful way to view this problem is epistemically: information-flow control is not only about where data moves, but also about what a public observer can learn about secrets from the outputs they are allowed to see.

This problem matters because many realistic systems must manipulate sensitive data while still producing useful public results. Login systems must reveal whether a password check succeeds. Statistical systems may need to release aggregate information, such as an average salary, without exposing each individual salary. Logging and auditing systems may need to reveal part of an identifier while keeping the rest secret. These examples show the central tension in the field: useful systems often need to release some information, but that release must be controlled so that the system does not reveal more than intended.

The literature on IFC has therefore developed along two connected tracks. One track asks how to define secure information flow precisely and enforce it through programming-language mechanisms. The other asks how to handle intentional release, often called *declassification*, without collapsing the security guarantees. Together, these questions have shaped several decades of work, beginning with mathematical models of secure flow, continuing through formal security properties and practical language designs, and leading to more recent attempts to simplify information-flow reasoning while still supporting realistic declassification policies.

1.2 Evolution of Results

A foundational starting point is Denning's lattice model of secure information flow [2]. Denning argues that security cannot be captured adequately by direct access control alone, because information can leak through chains of operations and control flow. Her key contribution is to model

security classes and information-flow constraints using a bounded lattice. This provides a clean mathematical structure for reasoning about when information may flow from one class to another. Denning’s work is important not only because it gives IFC an abstract foundation, but also because it points toward enforcement mechanisms such as runtime tracking and compile-time certification. In that sense, it establishes the core idea that secure flow must be treated as a first-class property of programs.

Goguen and Meseguer shift the discussion from mathematical structure to security policy itself [3]. They argue that the literature often blurs the distinction between a security policy and a model of the system being analyzed. Their notion of noninterference sharpens this distinction. They determine security as a claim that the actions of one user or command should have no observable effect on what another can see. This move is highly influential because it reframes IFC in semantic terms. Rather than only asking whether flows respect a lattice ordering, the field can ask whether high-security behavior interferes with low-security observation. Their work clarifies what later type systems and program logics are actually trying to guarantee.

Abadi, Banerjee, Heintze, and Riecke extend the conversation by asking whether secure information flow is just one instance of a more general notion of dependency [1]. Their Dependency Core Calculus (DCC) presents a shared foundation for several analyses, including IFC, slicing, binding-time analysis, and call tracking. This is a major conceptual step because it suggests that information-flow reasoning is not an isolated security trick, but part of a broader theory of how outputs depend on inputs. DCC also helps explain and compare earlier systems by showing how their guarantees can be translated into a common core. In the development of the field, this paper serves as a bridge between security-specific models and more general semantic accounts of dependency.

If DCC represents unification at the theoretical level, Myers’s JFlow represents an attempt to make IFC practical for programmers [5]. Earlier IFC languages were often too restrictive or too idealized to be used in realistic software. JFlow addresses this by extending Java with mostly static information-flow checking. It adds decentralized labels, statically checked declassification, label polymorphism, runtime label tests, and label inference, while also supporting more realistic language features such as objects, subclassing, and exceptions. JFlow is significant because it shifts the field from asking whether secure information flow can be formalized to asking whether it can be integrated into ordinary programming practice. It also makes the tension around declassification much more visible: once a system is expressive enough to model real applications, some controlled release becomes unavoidable.

That tension motivates the survey by Sabelfeld and Sands on declassification [6]. By 2005, the field had produced many partially connected definitions of intentional release, but it was often unclear how they related to one another. Sabelfeld and Sands organize this space using four dimensions: what information is released, who controls the release, where in the system release occurs, and when release is allowed. This paper is influential because it gives the field a vocabulary for comparing approaches that might otherwise seem unrelated.

A recent example of that next stage is the work of Gouni, Pfenning, and Aldrich on structural information flow [4]. Their paper argues that existing IFC systems have become too complex, especially once declassification is added as an extra feature. They revisit the foundations of the field and propose a system based on hybrid-logic ideas that tracks dependencies more directly. This also connects the paper to modal and hybrid logic, since it treats information flow in terms of explicit states and their relationships, rather than only as a system of label-checking rules. A central claim of the paper is that declassification need not appear only as an ad hoc exception to noninterference, instead, it can emerge naturally. This is an important modern direction because it responds directly to a long-standing weakness of the field: many systems become hard to use or hard to reason about when they try to support declassification.

Together, these papers show a clear progression. Denning gives the mathematical structure for secure flow. Goguen and Meseguer define a central security property in terms of noninterference. Abadi et al. place this reasoning within a broader theory of dependency. Myers brings IFC into a more realistic programming language setting. Sabelfeld and Sands organize the declassification problem that such practical systems inevitably face. Gouni et al. then revisit the foundations in an effort to simplify the theory while handling declassification more cleanly.

1.3 Open Research Questions

Despite decades of progress, several major research questions remain open. First, usability is still a persistent challenge. Even strong IFC systems can impose heavy annotation burdens or require programmers to think in unfamiliar ways about labels, policies, and authority. A recurring question is how to preserve strong guarantees while making such systems practical for ordinary software development.

Second, declassification remains unsettled. The literature now has a much richer vocabulary for describing release policies, but there is still no single framework that is both expressive and easy to enforce. Different systems emphasize different dimensions of release, and combining them cleanly remains difficult [6]. Relatedly, modern work such as structural information flow raises the question of whether declassification should be treated as a special mechanism at all, or whether it should emerge from more general reasoning principles [4].

Finally, many IFC systems still abstract away from important real-world leakage channels. Timing behavior, concurrency, resource usage, and other side channels can ruin otherwise elegant formal guarantees [5]. This creates a gap between the idealized guarantees proved by language-based systems and the messier behavior of real-world systems.

References

- [1] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proc. 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1999.

Problem: The paper asks whether several seemingly different dependency analyses, secure information flow, slicing, binding-time analysis and call tracking, actually share a common core notion of dependency. Their goal is to replace these separate arguments with one framework for reasoning about higher-order programs.

Solution: The authors introduce the Dependency Core Calculus (DCC), a small core language for dependency reasoning. They give it a denotational semantics, prove noninterference properties, and translate several existing calculi into DCC so those systems can inherit generic dependency results.

Evaluation: This is a theoretical paper. The authors evaluate DCC by defining its syntax, typing rules, and semantics, then showing that several existing dependency analyses can be translated into DCC so that their correctness results follow from DCC’s generic noninterference theorems.

Limitations: The paper does not claim that every dependency analysis fits directly into DCC. It also shows that plain DCC is not enough for some weaker call-by-value settings, which motivates the variant vDCC.

- [2] D. E. Denning, “A lattice model of secure information flow,” *Communications of the ACM*, vol. 19, no. 5, pp. 294–302, 1976.

Problem: Denning asks how to guarantee secure information flow, not just access control. She argues that access controls can miss indirect leaks through chains of operations and control flow. She seeks a mathematical framework that makes these security constraints systematic and decidable.

Solution: She introduces a formal model of information flow based on security classes and a flow relation, then shows that these classes should form a bounded lattice. This lattice gives a systematic way to reason about both explicit and implicit flows and suggests enforcement mechanisms such as run-time checks and compile-time certification.

Evaluation: This is a theoretical paper. Denning evaluates the model by showing that semantic consistency requirements on information flow justify the lattice structure and by explaining how the model supports enforcement through run-time mechanisms and program certification.

Limitations: The model does not address covert channels and does not capture every system security requirement. Denning also notes limits of certification: it cannot verify flows not represented in the program text, and hardware or language defects may still create insecure behavior.

- [3] J. A. Goguen and J. Meseguer, “Security policies and security models,” in *Proc. IEEE Symposium on Security and Privacy*, 1982, pp. 11–20.

Problem: The paper argues that many existing security models are too informal or too tied to particular systems. It also says security policy should be separated from the system model used to verify it.

Solution: The paper proposes an automaton-based framework and defines security policies as sets of noninterference assertions. Instead of listing all allowed flows directly, the policy states which users or commands should have no observable effect on others, and security verification becomes proving that a system model satisfies those assertions.

Evaluation: This is a theoretical paper. The authors evaluate the framework by formally defining system models, capability systems, and noninterference-based policies, then showing how security verification can be expressed as proving that the model satisfies the policy.

Limitations: The paper does not address user authentication, inference attacks, aggregation problems, or fault-tolerant secure computing. It also presents the framework as a foundation, suggesting future work for specialized verification tools.

- [4] H. Gouni, F. Pfenning, and J. Aldrich, “Structural information flow: A fresh look at types for non-interference,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, 2025.

Problem: The paper argues that existing information-flow systems are too complicated to scale cleanly to real programs, especially when declassification is needed. It asks whether the information-flow framework can be rebuilt on a simpler logical foundation that supports practical declassification without weakening reasoning.

Solution: It introduces structural information flow, a system based on hybrid-logic ideas that tracks dependencies more directly. In this framework, declassification is treated as something that can arise naturally from the logic rather than as a separate ad hoc escape hatch.

Evaluation: This is a theoretical paper. The authors evaluate the approach through formal definitions, worked examples, and a logical-relations proof of noninterference, showing that the system supports useful cases such as password checking while keeping reasoning local.

Limitations: The paper focuses on foundational type-theoretic reasoning rather than implementation in a real programming language. It suggests future work on connecting the approach to richer principal-based systems, studying its categorical semantics, and exploring ways to bring the style to existing languages.

- [5] A. C. Myers, “Jflow: Practical mostly-static information flow control,” in *Proc. 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1999.

Problem: The paper argues that earlier static information-flow languages were too restrictive to be practical. It asks how to make mostly static information-flow control usable in a realistic programming language.

Solution: Myers introduces JFlow, an extension of Java that treats information-flow checking as part of type checking. JFlow adds practical features such as decentralized labels, declassification, label polymorphism, run-time label checks, and label inference.

Evaluation: This is mainly a language-design and compiler paper. Myers evaluates JFlow by defining the language, presenting formal checking rules, and describing a source-to-source compiler strategy that keeps most checking static and therefore aims to keep run-time overhead low.

Limitations: JFlow is not fully backward compatible with Java and does not eliminate covert channels such as timing, threads, and some resource-based leaks. The paper also notes that giving full formal correctness proofs for a language with features like objects and inheritance is difficult.

- [6] A. Sabelfeld and D. Sands, “Declassification: Dimensions and principles,” *Journal of Computer Security*, 2005.

Problem: The paper argues that declassification research is fragmented across many partially connected definitions, making it hard to compare what different release policies actually guarantee.

Solution: Rather than proposing a new declassification mechanism, the paper organizes existing approaches along four dimensions: what information is released, who controls the release, where release happens, and when release is allowed. It also proposes semantic principles for judging declassification models.

Evaluation: This is a survey paper. The evaluation consists of comparing existing declassification approaches across the four dimensions and analyzing them using principles such as semantic consistency, conservativity, monotonicity of release, and non-occlusion.

Limitations: The paper does not propose a new enforcement mechanism, and its principles are presented as guidance rather than mandatory requirements. It points toward future work on combining the four dimensions into stronger overall policy frameworks.